

display window to the attribute `self.screen`, so it will be available in all methods in the class.

The object we assigned to `self.screen` is called a surface. A *surface* in Pygame is a part of the screen where a game element can be displayed. Each element in the game, like an alien or a ship, is its own surface. The surface returned by `display.set_mode()` represents the entire game window. When we activate the game's animation loop, this surface will be redrawn on every pass through the loop, so it can be updated with any changes triggered by user input.

The game is controlled by the `run_game()` method. This method contains a while loop ❸ that runs continually. The while loop contains an event loop and code that manages screen updates. An *event* is an action that the user performs while playing the game, such as pressing a key or moving the mouse. To make our program respond to events, we write an *event loop* to *listen* for events and perform appropriate tasks depending on the kinds of events that occur. The for loop ❹ nested inside the while loop is an event loop.

To access the events that Pygame detects, we'll use the `pygame.event.get()` function. This function returns a list of events that have taken place since the last time this function was called. Any keyboard or mouse event will cause this for loop to run. Inside the loop, we'll write a series of if statements to detect and respond to specific events. For example, when the player clicks the game window's close button, a `pygame.QUIT` event is detected and we call `sys.exit()` to exit the game ❺.

The call to `pygame.display.flip()` ❻ tells Pygame to make the most recently drawn screen visible. In this case, it simply draws an empty screen on each pass through the while loop, erasing the old screen so only the new screen is visible. When we move the game elements around, `pygame.display.flip()` continually updates the display to show the new positions of game elements and hide the old ones, creating the illusion of smooth movement.

At the end of the file, we create an instance of the game and then call `run_game()`. We place `run_game()` in an if block that only runs if the file is called directly. When you run this *alien_invasion.py* file, you should see an empty Pygame window.

Controlling the Frame Rate

Ideally, games should run at the same speed, or *frame rate*, on all systems. Controlling the frame rate of a game that can run on multiple systems is a complex issue, but Pygame offers a relatively simple way to accomplish this goal. We'll make a clock, and ensure the clock ticks once on each pass through the main loop. Anytime the loop processes faster than the rate we define, Pygame will calculate the correct amount of time to pause so that the game runs at a consistent rate.

We'll define the clock in the `__init__()` method:

alien_invasion.py

```
def __init__(self):
    """Initialize the game, and create game resources."""
    pygame.init()
```